ML Lab Endsem

Madhav Arun Krishna - DA24B012

November 2025

1 Introduction

For this endsem project, I began by trying out some data preprocessing techniques like PCA and Outlier Detection with LOF, and then I tried several models like XGBoost, Kmeans, K-Nearest neighbours, Softmax, 1v1 Logistic Regression.

We have performed bias variance trade-off analysis, hyperparam tuning analysis, used boosting for our XGBoost model, used a voting ensemble for merging models, and using different models for a data depending on the predicted probabs and consistency of prediction across models.

I have included my analysis for everything below in this report.

2 Data Pre-Processing

There are some techniques I tried to use before training the main models (which we will discuss in the next section)

1. PCA (Principal Component Analysis)
2. Outlier Detection with LOF (Local Outlier Frontier)

In the following subsections, I will talk about my results and analysis I did to find the optimal preprocessing required:

2.1 PCA (Principal Component Analysis)

- The key idea of PCA is that real world data is that real world data is usually of a much lower dimension than it appears in
- So for example, our 28x28 image data from MNIST is technically 784 dimensions (1 dim for each pixel), but in reality, there exists a much lower dimensional subspace that contains basically the same information.
- For this, we will calculate the Singular Value Decomposition of $X^T X$ where X is your training X matrix

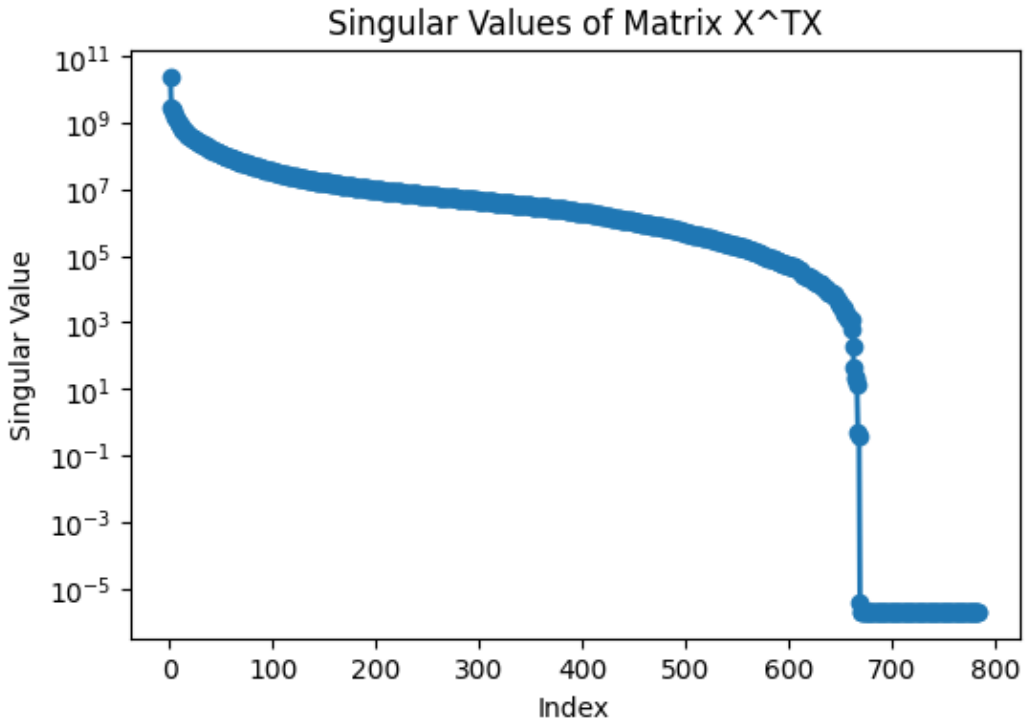


Figure 1: Magnitude of all the singular values in MNIST Train

Note that the Y-axis on the log scale

- We can see that the first 60 or so Singular Values are almost 10x the magnitude of the rest.
- After that, there is a gradual decline and after 668 singular values or so, the magnitude drops significantly, dropping to 10^{-5}, practically becoming useless.

We will take advantage of the fact that the magnitude of the singular values drop after 60 for our future models.

2.2 Outlier Detection with LOF (Local Outlier Frontier)

We can use outlier detection to get rid of outliers using the LOF method that we were taught. There are couple of things to note though on my testing:

- I ran outlier detection on the data and got that there are approximately 12 / 10002 outliers in the full training datasest keeping K = 5. This would obviously change depending on the threshold
- This process is very slow (there could be speedups using advanced techniques, but this is still relatively slow given the 5 minute time constraint)
- Now normally we would remove these outliers from the training data before training to avoid misleading the model, but since LOF is very slow, and since the outliers are very small in number, i excluded them from the final ensembled model.

3 All Tried Models

I tried several different models:

1. XGBoost
2. Kmeans Clustering

3. K-Nearest Neighbours
4. Softmax Regression
5. 1v1 Logistic Regression

In the following subsections, I will talk about my thought processes, and some results and accuracies on them. (This is just the individual models and their performances, I will talk about the final stacked/ensembled model later)

3.1 XG Boost

I modified my assignment 2 submission code for the XGBoost model to allow it to be a multi-class classifier. Here are the highlights of the changes made from the assignment 2 submission:

- The core logic for the XGBoost Trees stays the same (where every tree still is a binary predictor
- The difference is that we are now training 10 trees for every n, and each train gives a predicted probability. At every iteration of n, the boosting algorithm will train the next iteration based on the errors of the previous iterations up till that point.
- At the end of the whole process, when predicting, we will apply a softmax for deciding which class has the highest probab and select that as the prediction.

On this new XGBoost, I ran hyper-parameter tuning to get the best possible accuracy. In order to perform the hyperparameter tuning, I fixed all the parameters and varied just one param to see how the model performance depends on that hyperparameter. Here are my observations:

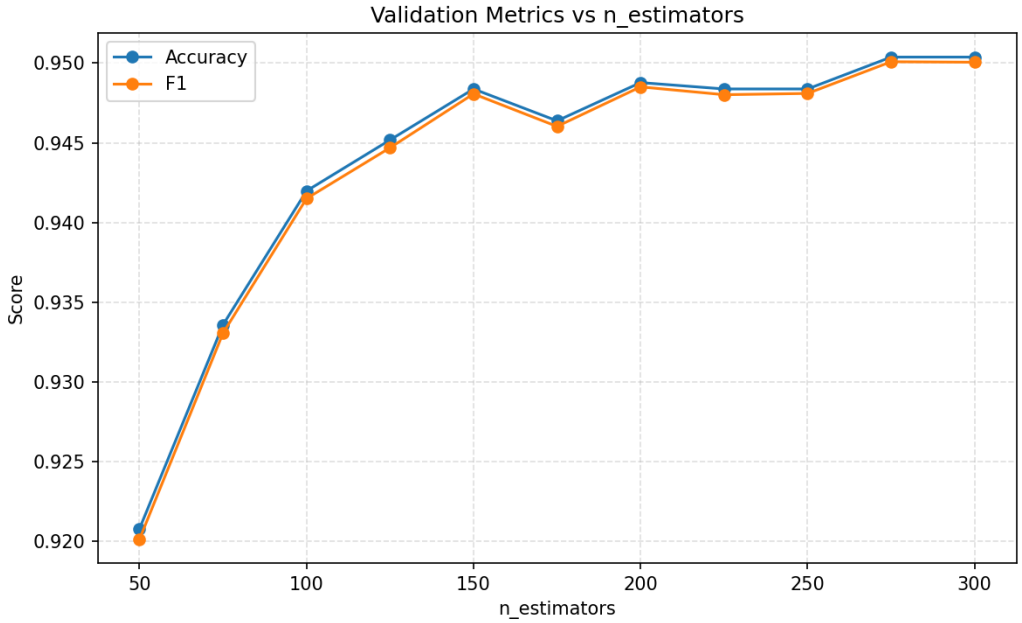


Figure 2: Enter Caption

N Estimators
We see that increasing the number of estimators clearly leads to increase in performance but at some point it begins to saturate. (Very similar to the previous assignment as expected). For the final model, i decided to pick $n = 250$ as it begins plateauing after that, and increasing n will significantly increase the training time.

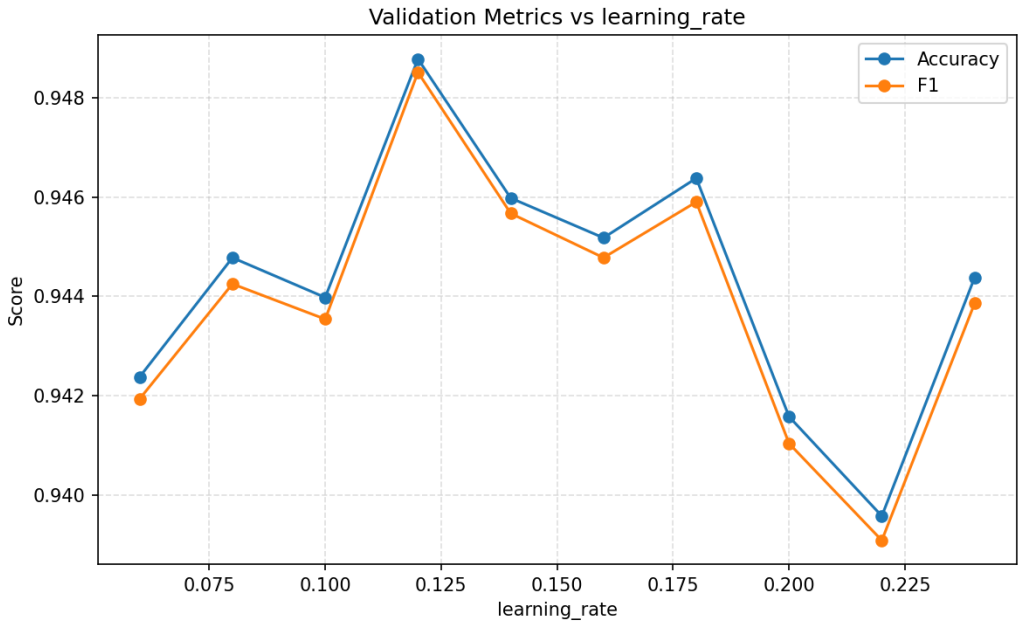


Figure 3: Variation of Acc/F1 wrt Learning Rate

Learning Rate

We see that low ratings means that the model takes too long to learn, and we need to increase n to compensate which increases training time, while very high learning reates struggle to converge close to the answer and bounce around a lot, leading to lower acc and F1. We see the best lr seems to be around 0.12, which is the lr I selected for the final model.

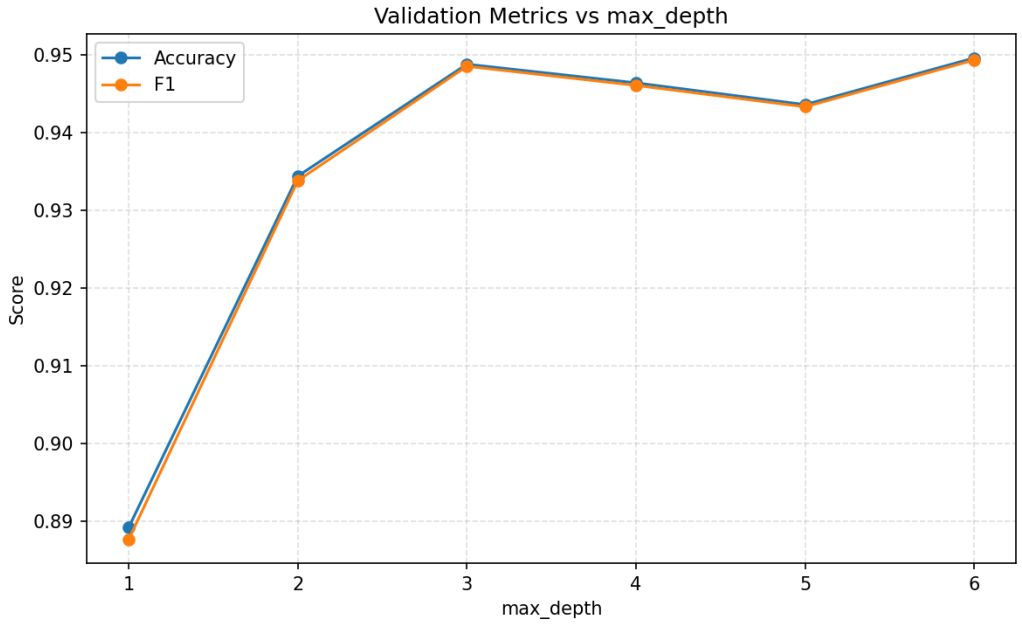


Figure 4: Variation of Acc/F1 wrt Max Depth

Max Depth

Increasing max depth increases the training time quite significantly as the trees have to much deeper. We see that beyond max depth = 3, there is almost no benefit of increasing it, so we can get away with keeping max depth = 3 and maximize how much acc we can get in the training time constraints.

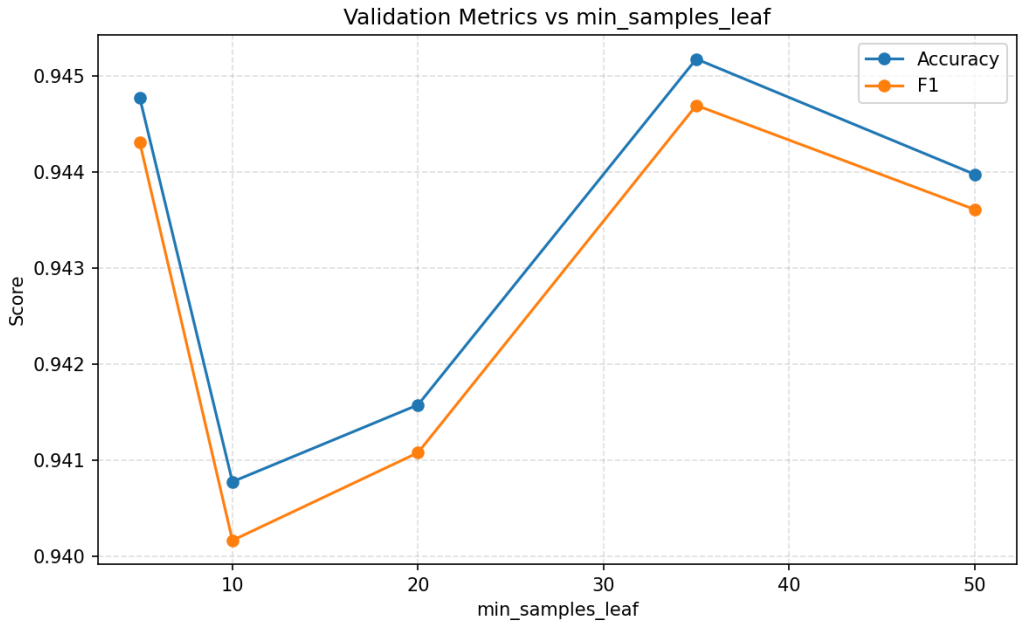


Figure 5: Variation of Acc/F1 wrt Min Samples per leaf

Min Samples per leaf

In this case, the variation in the y axis is quite small (Only 0.005) from top to bottom. This is because our max depth is pretty small, the min samples per leaf is not really getting hit, which makes sense. Hence this value doesnt really matter too much.

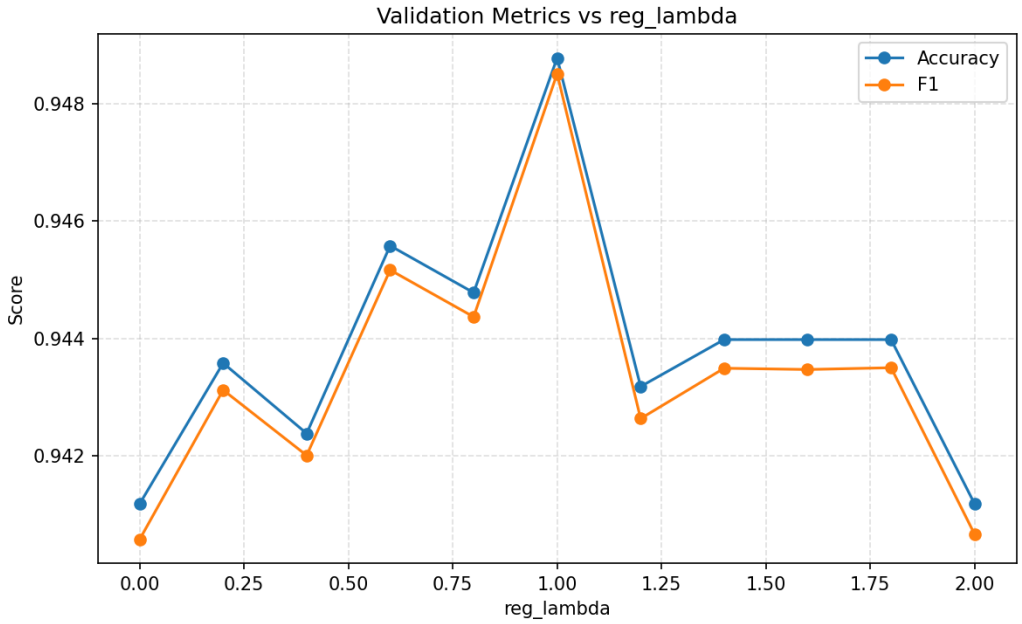


Figure 6: Variation of Acc/F1 wrt Regularization Lambda

Regularization Lambda

We see that there is a clear noticable best lambda here which is 1. Increasing or decreasing seems to reduce the model performance (Although quite insignifantly as the y axis is just differing by 0.006). We will still stick to the best lambda of 1.

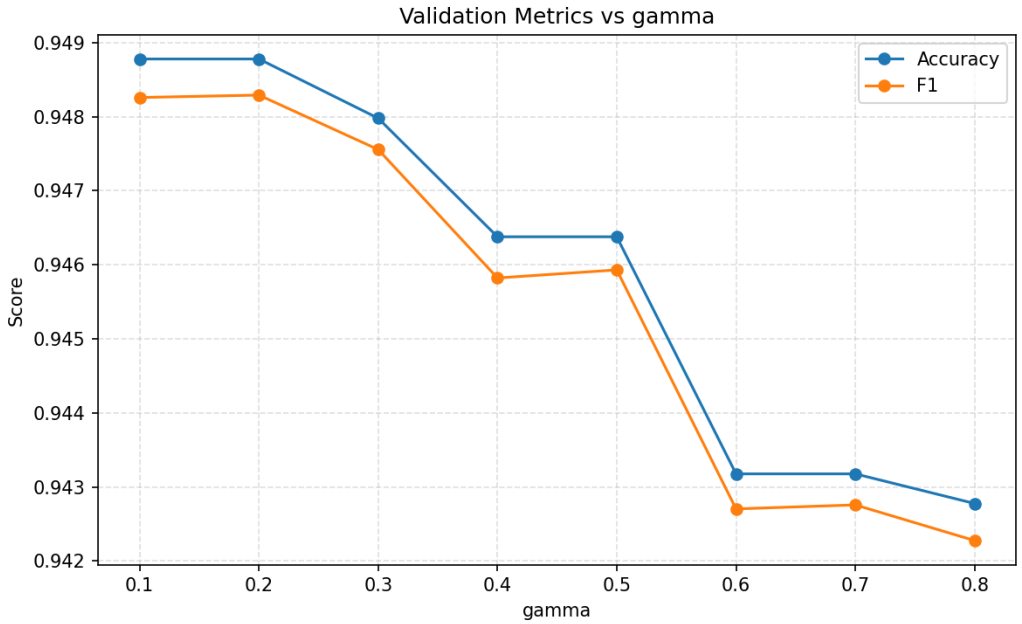


Figure 7: Variation of Acc/F1 wrt Gamma

Gamma

Increasing gamma generally helps prevent overfitting, but if we increase it to very high values, we see that the accuracy is affected too much. Thus it is better to keep the gamma slightly lower, so I decided to use around 0.25

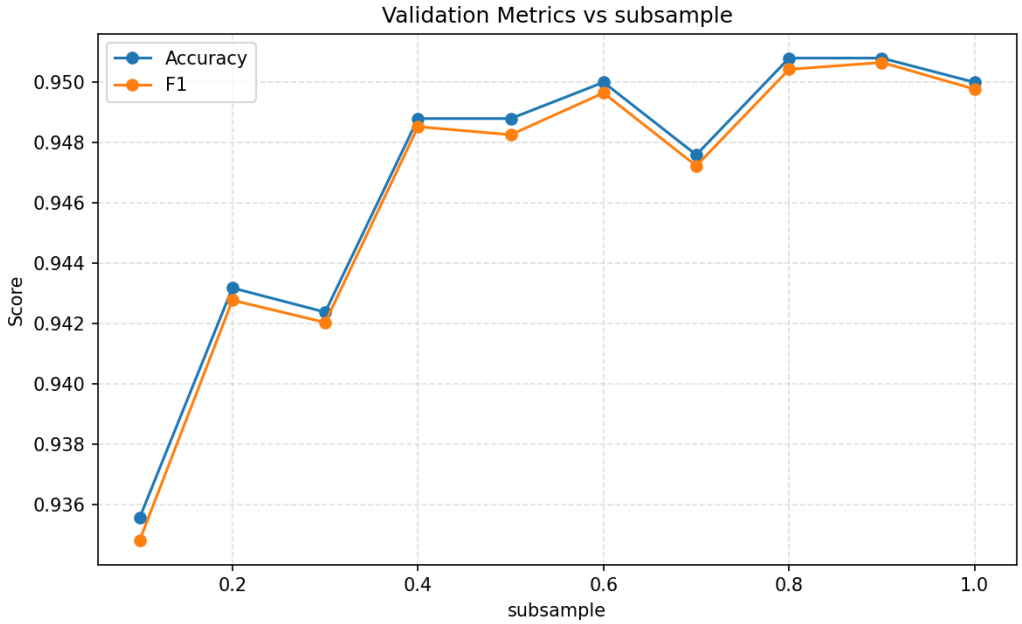


Figure 8: Variation of Acc/F1 wrt Row Subsample Fraction

Row Subsample Fraction

Decreasing the row subsample fraction will help reduce the training time. So we should make it as small as possible that still preserves the accuracy. In this case, that seems to be around 0.4, which is what I used in my model

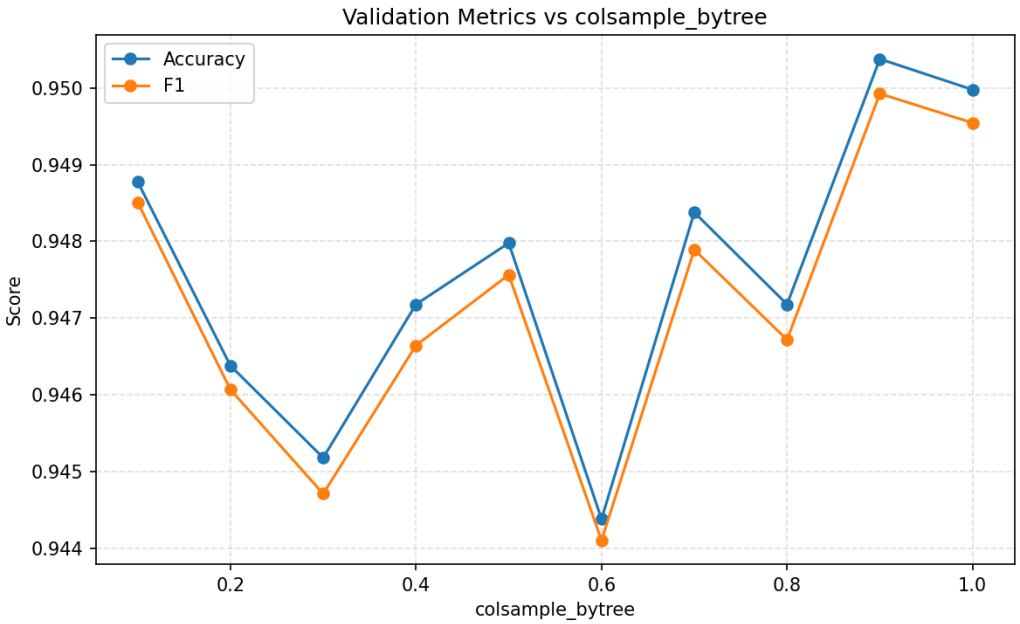


Figure 9: Variation of Acc/F1 wrt Column Subsample Fraction

Column Subsample Fraction
Decreasing the column subsample fraction will help reduce the training time. So we should make it as small as possible that still preserves the accuracy. Here, 0.1 seems to be doing just as good as higher values (of course it is still worse that 0.9, but because it is 9 times faster approximately, it is far better for us.)

Trying PCA with XGBoost:
When I tried PCA with XGBoost, it seemed to negatively affect performance. Hence I decided to not use PCA for XGBoost. I still use it for another model.

Final Metrics with just XGBoost for our given hyperparams:

- Accuracy: 0.9483
- F1: 0.9481

This is already great. But I tried using more models and ensembling them to get a better accuracy.

3.2 Kmeans Clustering

Although clustering is usually only used for unsupervized data, we can use it for supervized data by making several clusters, and then labelling all the points in the cluster by the same label (the majority label).
When trying this, I noticed that the performance was quite bad (less than 80% accuracy). Now I tried tuning some hyperparameters etc, but without significantly increasing the training time, I was not able to get anything conclusive. Hence I decided to move on to the next model without including this in our final ensembled model.

3.3 K-Nearest Neighbours

This is a much better method for classification based tasks as compared to K-means as it is designed to work for labeled data. Firstly, I tried raw KNN without any PCA (I tried finding the best k):

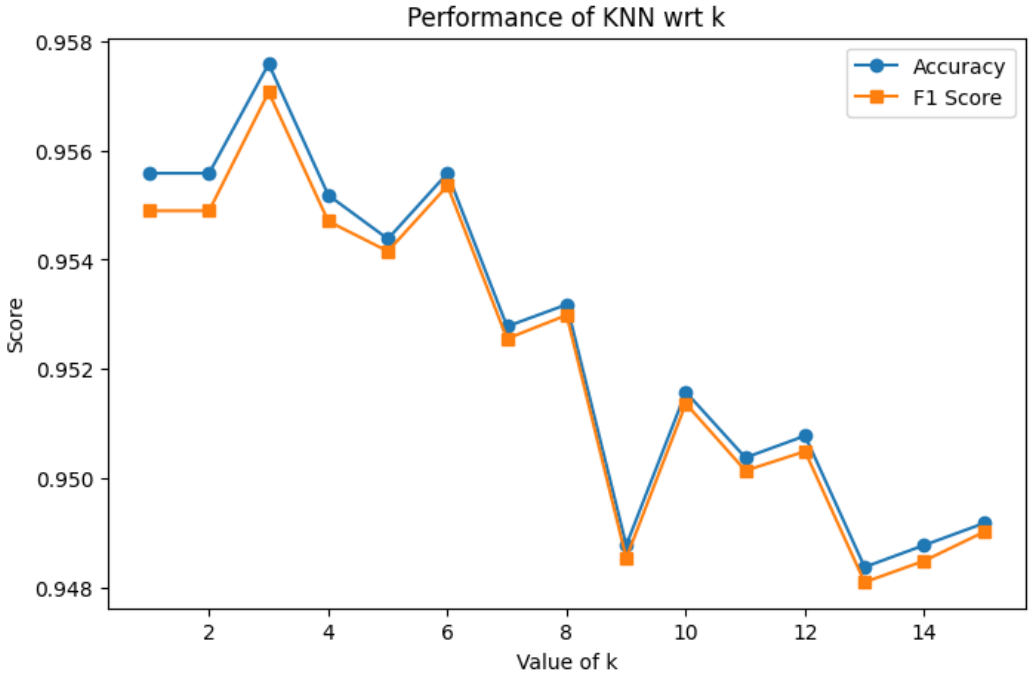


Figure 10: Finding the best k for KNN

This plot is a very good example of the **bias-variance trade-off**. We see that as k increases in the beginning from 1 to 3
We see that the accuracy and F1 score is the best for $k = 3$, and hence I included this in my final model.

- Accuracy: 0.95758
- F1 Score: 0.95706

I also wanted to try using PCA before feeding into the KNN, and hence obviously, we need to figure out how many components I should include from PCA. From the previous analysis on PCA (Discussed in the previous section), we had noticed that $n = 60$ was a good number of PCA components after analysing the PCA graph.

But I deicded to still try and plot the accuracy against the number of PCA components if we trained a KNN after applying PCA

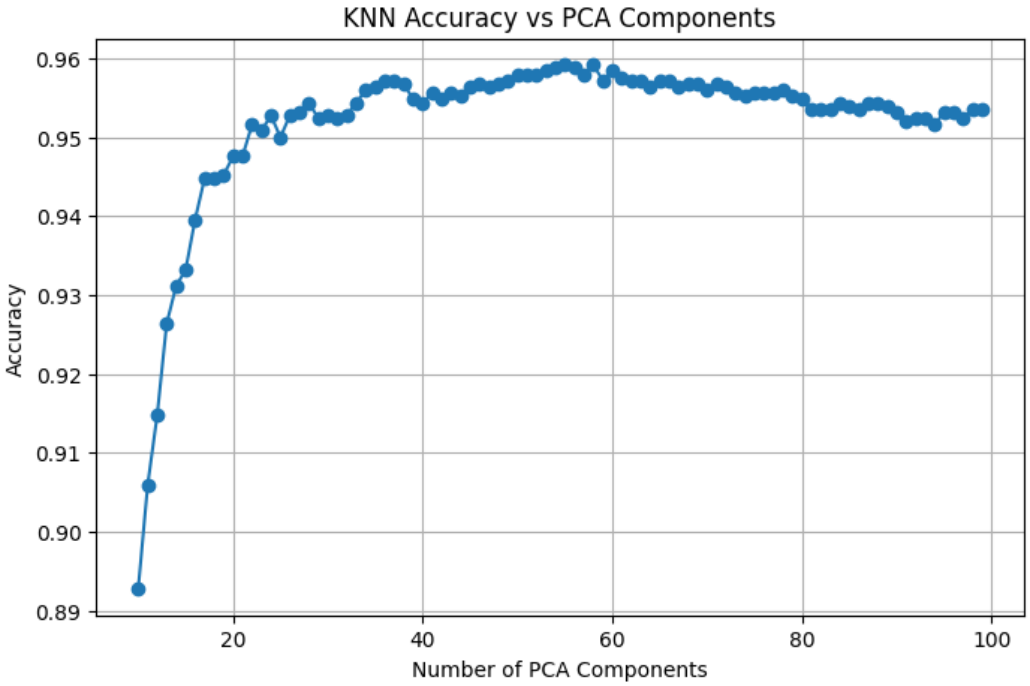


Figure 11: Finding the best number of PCA components for KNN

In this case, we see that our original hunch was right, with the accuracy saturating around 60, but interestingly, we see the accuracy begins dropping after that (although very insignificantly) - this could be because of inclusion of noise from lower priority PCA dims that confuse the model on some edge cases.

- Accuracy: 0.95838
- F1 Score: 0.95809

3.4 Softmax Regression

This is the most classic simple model where we just apply a simple softmax. This is the most simple and natural extension of logsitic regression into a multi class problem. I thought I'd try this as well. It doesn't seem to perform very well:

- Accuracy: 0.8864
- F1: 0.8833
- Train Time: 1.8s

Although it is very fast, its accuracy is much lower than the KNN and XGBoost we discussed, and hence we wont include in the final model.

3.5 1v1 Logistic Regression

The idea here is that we convert the multiclass problem into a binary classification problem, where every individual model only needs to predict if an element belong to class i or class j. It is just used to compare these 2. The idea is that, if we only train it on 2 classes, it should get really good at distinguishing them.

So obviously, we would need to train ${}^{10}C_2 = 45$ models for all the 10 classes. But because logistic regression models are very fast in training, and because we only need to train it on the relevant training set and not the full train set it is very fast.

Here are the accuracies after training:

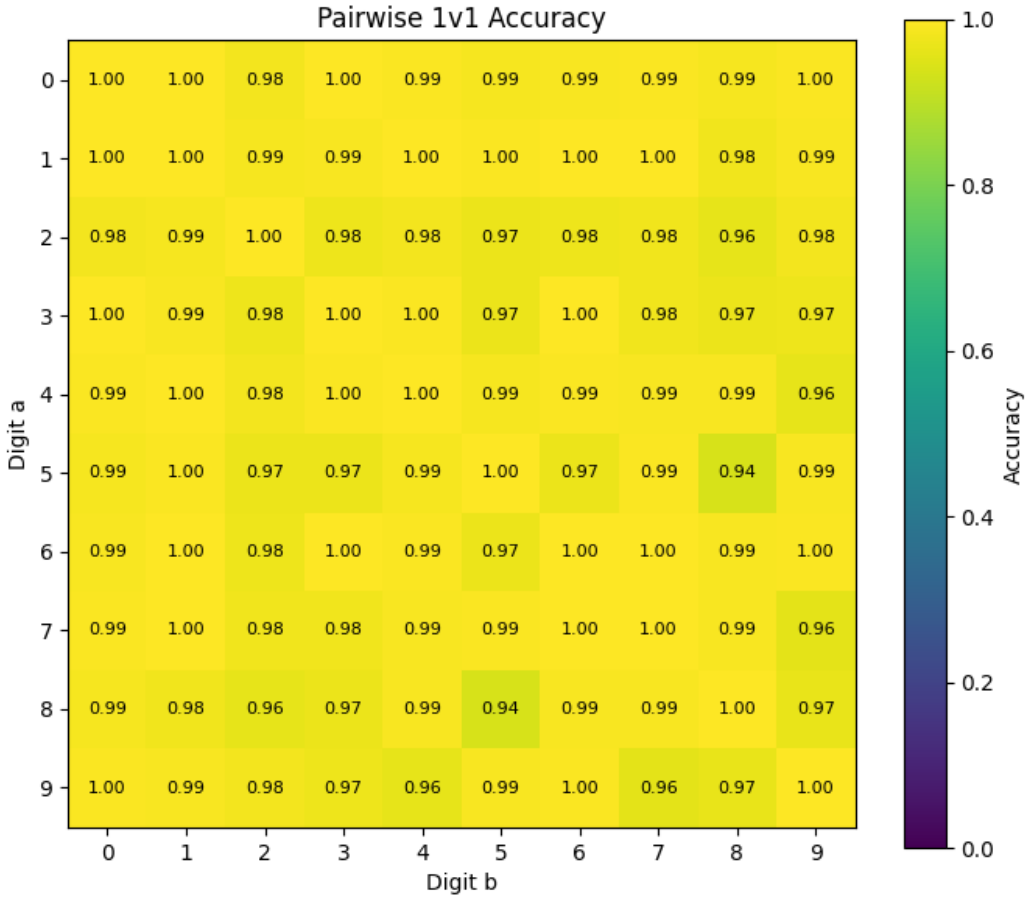


Figure 12: 1v1 Accuracy

- We see that this model is super accurate, excluding a few confusing digits like 5 and 8.
- The issue with this model is this accuracy only works if we know before hand which of the 2 classes it belongs to. If we try taking a test point and run it through all these models, we would need to rely on predicted probabs to compare, which will reduce accuracy
- Hence, I will use this model to predict situations of disagreement between other models in my ensembled model (So if KNN is predicted 1, and Xgboost is predicting 2, I will use this model to predict between 1 and 2 as a tie breaker situation) (More details later on)

4 Final Ensembled Pipeline

Here is the final pipeline:

- Since XGBoost and KNN performed the best, we will use these 2 as our primary predictors.
- We will take the training data and train our XGBoost on the final parameters chosen
- We will take our training data and directly apply to a KNN with k = 3
- We will take our training data and apply PCA, and train a separate KNN
- We will also train all 45 1v1 Logistic Regression Models

On prediction:

- We will pass the data into our KNN, our PCA-KNN and XGBoost and run a voting classifier ensembler on this.
- If there is a majority vote available (≥ 2 models agree on the solution), we will directly pass that solution to our final submission.
- If all three models are ambiguous, we will run the 1v1 across all three of those solutions through our 1v1 models. The results are added to the final prediction.

I am also presenting a flow chart of my solution pipeline:

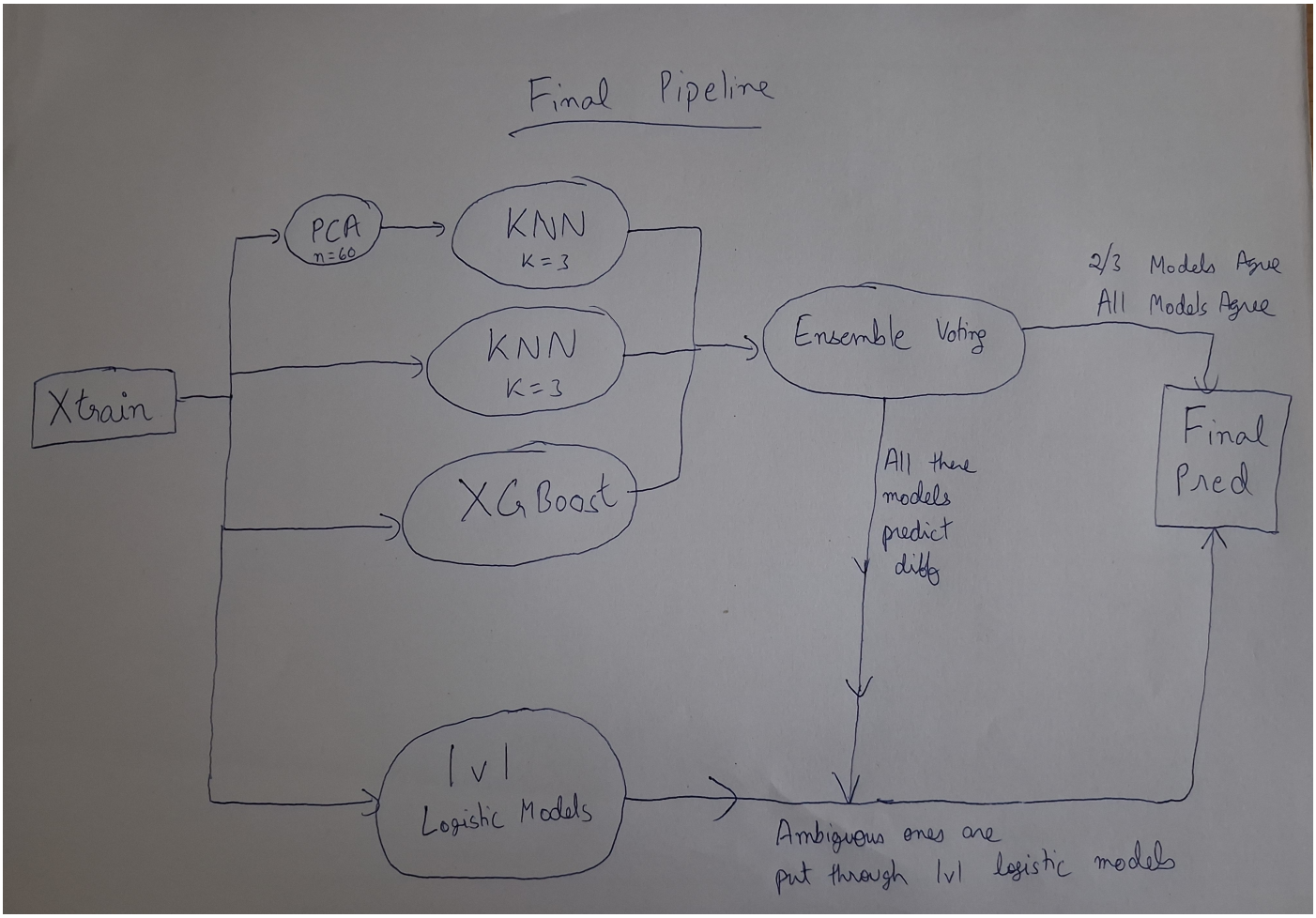


Figure 13: Final Pipeline

Metrics and Results for the Final ensembled model on Validation Data:

- Accuracy: 0.9628
- F1 Score: 0.9624

We can see that this ensemble was able to outperform all the individual previous models and break the 96% threshold on the validation data.
Here is the confusion matrix for the final ensemble:

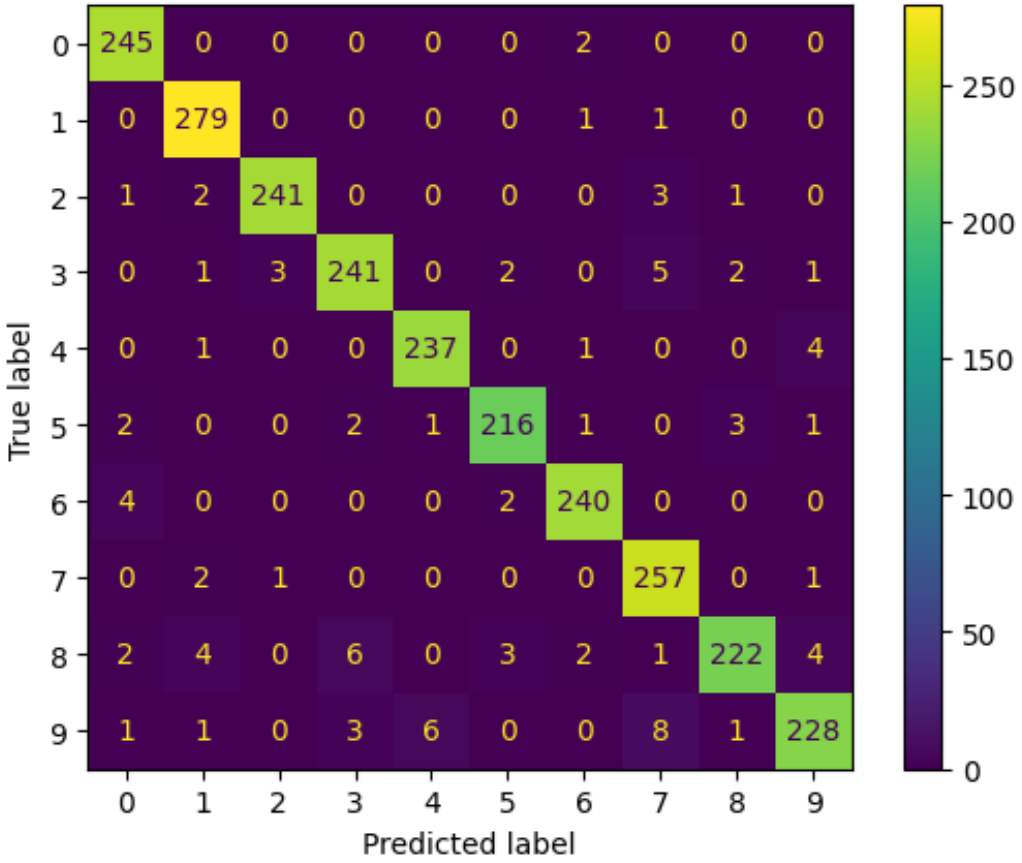


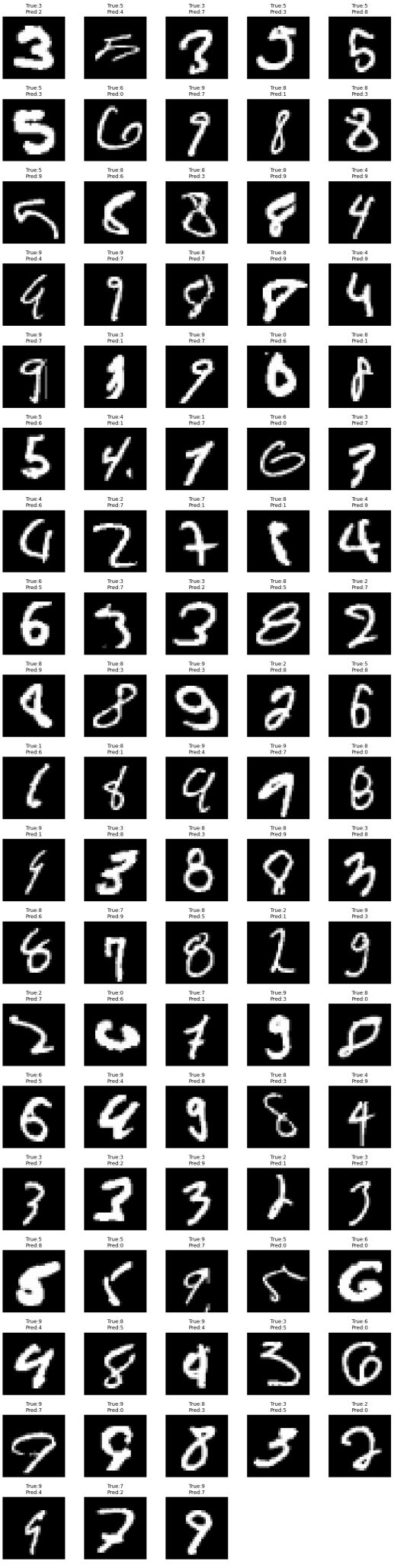
Figure 14: Confusion Matrix on Final Ensemble

Here is the tabulation of my efforts:

Table 1: Summary of validation performance for all models

Model	**Accuracy**	**F1 Score**
XGBoost (tuned)	0.9483	0.9481
KNN (k = 3)	0.9576	0.9571
KNN + PCA (60 comps)	0.9584	0.9581
Kmeans Clustering	0.8242	0.8123
Softmax Regression	0.8864	0.8833
1v1 Logistic Regression	(In Pairwise matrix)	(In Pairwise matrix)
LOF	(Negligible Benefits)	(Negligible Benefits)
Final Ensemble	**0.9628**	**0.9624**

In the validation set we totally misclassified 93/2499, which is a very small fraction. I chose to visualize these to see what kind of numbers the model is struggling with:


True:3 Pred:2
True:5 Pred:4
True:3 Pred:7
True:5 Pred:3
True:5 Pred:8
True:5 Pred:3
True:6 Pred:0
True:9 Pred:7
True:8 Pred:1
True:8 Pred:3
True:5 Pred:9
True:8 Pred:6
True:8 Pred:3
True:8 Pred:9
True:4 Pred:9
True:9 Pred:4
True:9 Pred:7
True:8 Pred:7
True:8 Pred:9
True:4 Pred:9
True:9 Pred:7
True:3 Pred:1
True:9 Pred:7
True:0 Pred:6
True:8 Pred:1
True:5 Pred:6
True:4 Pred:1
True:1 Pred:7
True:6 Pred:0
True:3 Pred:7
True:4 Pred:6
True:2 Pred:7
True:7 Pred:1
True:8 Pred:1
True:4 Pred:9
True:6 Pred:5
True:3 Pred:7
True:3 Pred:2
True:8 Pred:5
True:2 Pred:7
True:8 Pred:9
True:8 Pred:3
True:9 Pred:3
True:2 Pred:8
True:5 Pred:8
True:1 Pred:6
True:8 Pred:1
True:9 Pred:4
True:9 Pred:7
True:8 Pred:0
True:9 Pred:1
True:3 Pred:8
True:8 Pred:3
True:8 Pred:9
True:3 Pred:8
True:8 Pred:6
True:7 Pred:9
True:8 Pred:5
True:2 Pred:1
True:9 Pred:3
True:2 Pred:7
True:0 Pred:6
True:7 Pred:1
True:9 Pred:3
True:8 Pred:0
True:6 Pred:5
True:9 Pred:4
True:9 Pred:8
True:8 Pred:3
True:4 Pred:9
True:3 Pred:7
True:3 Pred:2
True:3 Pred:9
True:2 Pred:1
True:3 Pred:7
True:5 Pred:8
True:5 Pred:0
True:9 Pred:7
True:5 Pred:0
True:6 Pred:0
True:9 Pred:4
True:8 Pred:5
True:9 Pred:4
True:3 Pred:5
True:6 Pred:0
True:9 Pred:7
True:9 Pred:0
True:8 Pred:3
True:3 Pred:5
True:2 Pred:0
True:9 Pred:4
True:7 Pred:2
True:9 Pred:7